Studio Nx

JS 315 — 로딩 화면 연결과 샘플 미리 받기

2026-10-21

JavaScript 315 — 로딩 화면 연결과 샘플 미리 받기 (`sequencer.js`, `sequencer-audio.js`, `sequencer-devices.js`)

항목	내용
파일	`studionx/frontend/static/js/pages/sequencer.js`, `studionx/frontend/static/js/pages/sequencer-audio.js`, `studionx/frontend/static/js/pages/sequencer-devices.js`, `docs/handoff/p1-28_tests/lib.mjs`
작업	S1-14 시퀀서 로딩 화면: 로딩 화면에 진행 상황을 알려 주고, 프로젝트를 열 때 샘플 파일을 미리 받아 둠
작성	Claude (AI), 2026-10-21, 사용자 검토
이전, 다음	이전 314 시퀀서 로딩 화면, 다음 없음 (S1-14의 마지막 문서)

1. 하는 일

314가 로딩 화면을 그린다면, 이 문서는 그 화면에 "무엇을 얼마나 불러왔는지"를 알려 주는 쪽입니다. 로딩 화면이 보여 주는 진행률이 실제 일의 진행이어야 하므로, 프로젝트를 열 때 샘플 파일을 미리 받아 둡니다.

```
sequencer.js init()
  loaderMode → createLoader              로딩 화면을 붙잡음 (종류 정하기)
  projectId 가 없음 —▶ openWithLoader(예시)
  requireLogin → loadProject(GET /full) —▶ data
  openWithLoader(loader, data, mode)
     setProject(제목, 정보 글)             PROJECT DATA 완료
     open 이면:  projectSampleIds(data) —▶ 샘플 번호들 (클립과 Sampler, 중복 없이)
                 preloadSamples(ids, onStep) —▶ 한 번에 3개씩 받아 두고, 끝날 때마다 "n/m" 갱신
     new, demo:  불러올 샘플이 없으므로 바로 다음 단계
     devicesReady()                      트랙과 장치 화면이 그려질 때까지 기다림
     loader.finish()                     100% → SIGNAL IN → 열림
  (재생 또는 내보내기가 처음 샘플을 달라고 하면 loadSampleBytes 가 미리 받아 둔 것을 한 번 넘겨줌)
```

미리 받는 이유는 둘입니다. 진행률이 의미 있는 숫자("샘플 12/34")가 되고, 사용자가 재생을 처음 누를 때 샘플을 받느라 기다리지 않습니다. 이전에는 재생을 누르는 순간에 샘플을 받았습니다.

2. 언제 쓰나

쓰는 곳	하는 일
시퀀서 페이지를 열 때	`init`이 로딩 화면을 만들고 `openWithLoader`로 마무리합니다.
재생을 처음 누를 때	`PlaybackSession`이 `loadSampleBytes`를 부르고, 미리 받아 둔 샘플을 넘겨받습니다.
내보내기를 처음 할 때	`exportSnapshot().loadSampleBytes`가 같은 함수를 부릅니다.

3. 코드

3.1 시작과 마무리 (`sequencer.js`)

```
export async function init() {
  initHistory({ toast: showToast });
  initSave({ toast: showToast });
  initPalette({ toast: showToast });
  initShortcuts();
  initPatternCommands();
  setDeviceNotifier(handleChange);

  const projectId = readProjectId();
  const mode = loaderMode(window.location.search, !!projectId);
  const loader = createLoader({ mode: mode });
  if (!projectId) {                          // 예시 화면: 숫자·트랙을 그대로 두고 저장도 하지 않습니다
    await openWithLoader(loader, null, mode);
    return;
  }
  setProjectId(projectId);
  if (!(await requireLogin())) return;       // 로그인 안 됐으면 home.html 로 보냄 (이동하므로 로딩 화면은 그대로 둠)
  loader.setStage('data', 0.3);
  let data = null;
  try {
    data = await loadProject(projectId);
    if (data === null) return;               // 오류 화면(404 · 403 · 500)으로 이동 중 — 로딩 화면은 그대로 둠
  } catch (err) {
    if (err instanceof ApiError && err.status === 401) return;   // api.js 재발급 실패 → requireLogin 흐름
    loader.fail();                           // 기다리지 않고 바로 열고, 안내만 띄움
    showToast('Could not open this project — showing the example screen');
    if (window.console) console.warn('[sequencer]', err);
  }
  await openWithLoader(loader, data, mode);
}
```

```
async function openWithLoader(loader, data, mode) {
  try {
    if (!data && mode === 'demo') {
      // 예시 화면은 프로젝트가 없으므로 제목과 안내를 직접 적음 (저장되지 않는 화면이라는 것을 알림)
      loader.setProject({ title: 'Example Project', info: 'SAMPLE DATA, NOTHING IS SAVED' });
    }
    if (data) {
      const tracks = data.tracks || [];
      loader.setProject({
        title: data.project ? data.project.title : '',
        info: projectInfoText(data.project, tracks.filter(function (t) { return t.track_type !== 'MASTER'; }).length, mode),
      });
    }
    loader.setStage('data', 1);
    if (data && mode === 'open') {
      const ids = projectSampleIds(data);
```

```
      loader.setSampleTotal(ids.length);
      await preloadSamples(ids, function (done, total) { loader.setSamples(done, total); });
    } else {
      loader.setStage('content', 1);
    }
    loader.setStage('engine', 0.5);
    await devicesReady();
    if (typeof window.AudioContext === 'undefined' && typeof window.webkitAudioContext === 'undefined')
{
      showToast('This browser does not support Web Audio — playback and export are unavailable');
    }
    loader.setStage('engine', 1);
  } catch (err) {
    loader.fail();
    if (window.console) console.warn('[sequencer] loading screen', err);
  }
  await loader.finish();
}
```

3.2 샘플 미리 받기 (`sequencer-audio.js`)

```
function fetchSampleBytes(sampleId) {
  return fetchSampleFileUrl(sampleId).then(function (url) { return fetch(url); })
    .then(function (res) { return res.arrayBuffer(); });
}
```

```
function loadSampleBytes(sampleId) {
  const key = String(sampleId);
  const hit = preloaded[key];
  if (hit) {
    delete preloaded[key];
    preloadedBytes -= hit.byteLength;
    return Promise.resolve(hit);
  }
  return fetchSampleBytes(sampleId);
}
```

```
function preloadOne(sampleId) {
  return new Promise(function (resolve, reject) {
    let settled = false;
    const timer = window.setTimeout(function () {
      if (settled) return;
      settled = true;
      reject(new Error('timeout'));
    }, PRELOAD_TIMEOUT_MS);
    fetchSampleBytes(sampleId).then(function (bytes) {
      if (settled) return;
      settled = true;
      window.clearTimeout(timer);
      if (preloadedBytes + bytes.byteLength <= PRELOAD_CAP_BYTES) {
        preloaded[String(sampleId)] = bytes;
        preloadedBytes += bytes.byteLength;
      }
      resolve();
    }, function (err) {
```

```
      if (settled) return;
      settled = true;
      window.clearTimeout(timer);
      reject(err);
    });
  });
}
```

```
export function projectSampleIds(data) {
  const trackIds = ((data && data.tracks) || []).map(function (t) { return t.track_id; });
  return collectSampleIds(data ? data.clips : [], samplerSampleIds(playbackInstruments, trackIds));
}
```

```
export function preloadSamples(ids, onStep) {
  const list = (ids || []).slice();
  const total = list.length;
  const step = onStep || function () {};
  return new Promise(function (resolve) {
    if (!total) { resolve({ total: 0, loaded: 0, failed: 0 }); return; }
    let next = 0;
    let done = 0;
    let failed = 0;
    function finishOne(ok) {
      done += 1;
      if (!ok) failed += 1;
      step(done, total);
      if (done === total) resolve({ total: total, loaded: total - failed, failed: failed });
      else pump();
    }
    function pump() {
      if (next >= total) return;
      const id = list[next];
      next += 1;
      preloadOne(id).then(function () { finishOne(true); }, function () { finishOne(false); });
    }
    for (let i = 0; i < Math.min(PRELOAD_PARALLEL, total); i += 1) pump();
  });
}
```

```
export function preloadState() {
  return { count: Object.keys(preloaded).length, bytes: preloadedBytes };
}
```

3.3 화면 준비를 기다리는 약속 (`sequencer-devices.js`)

```
export function devicesReady() {
  return readyPromise;
}
```

`initDevices`가 플러그인 목록과 트랙을 읽어 그리는 연쇄(`loadPlugins`, `loadTracks`, `renderTracks`)를 `readyPromise`에 담아 둡니다. 로딩 화면은 이 약속이 풀릴 때까지 기다린 뒤에 엽니다. 그래야 로딩 화면이 걷혔을 때 트랙이 이미 그려져 있습니다. 읽기에 실패해도 약속은 풀립니다(실패 처리는 기존 `catch`가 합니다).

4. 실패해도 열립니다

시퀀서가 열리지 않는 것이 가장 나쁜 결과라서, 어떤 단계가 실패하거나 늦어도 로딩 화면은 열립니다.

상황	동작
샘플 하나를 받지 못함	끝난 것으로 세어 `3/3`까지 갑니다. 그 샘플은 담아 두지 않고, 재생할 때 다시 받습니다.
샘플 하나가 20초 넘게 걸림	건너뛰고 나머지를 계속합니다 (`PRELOAD_TIMEOUT_MS`).
전체가 25초를 넘김	그대로 엽니다 (`HARD_TIMEOUT_MS`, 314).
프로젝트(`GET /full`)를 읽지 못함	`loader.fail()`로 `NO SIGNAL`을 그대로 두고 바로 열며, "Could not open this project" 알림을 띄웁니다.
서버가 404, 403, 500을 줌	오류 화면으로 이동하는 중이라 로딩 화면을 그대로 둡니다 (`loadProject`가 `null`을 돌려줌).
로그인이 풀림	로그인 화면으로 이동하는 중이라 그대로 둡니다.
스크립트가 아예 멈춤	CSS 안전장치가 30초 뒤에 화면을 풀어 줍니다 (314).

5. 메모리

미리 받은 샘플은 재생이나 내보내기가 가져가기 전까지 메모리에 있습니다(`ArrayBuffer`). 큰 프로젝트에서 메모리를 오래 잡지 않도록 두 가지를 했습니다.

- 가져갈 때 한 번만 넘겨주고 지웁니다. `decodeAudioData`가 받은 버퍼를 비우므로 같은 버퍼를 다시 쓸 수도 없습니다.
- 담아 두는 총량의 한도가 256MB입니다(`PRELOAD_CAP_BYTES`). 넘는 샘플은 받기는 하되 담지 않고 버려서, 그 샘플만 재생할 때 다시 받습니다.

6. 최신 문법으로 쓰면

```
// preloadSamples 를 최신 문법으로 쓴 모양 (동시에 3개씩)
export async function preloadSamples(ids = [], onStep = () => {}) {
  const queue = [...ids];
  let done = 0, failed = 0;
  const worker = async () => {
    while (queue.length) {
      const id = queue.shift();
      try { await preloadOne(id); } catch { failed += 1; }
      onStep(++done, ids.length);
    }
  };
  await Promise.all(Array.from({ length: Math.min(3, ids.length) }, worker));
  return { total: ids.length, loaded: ids.length - failed, failed };
}

const hit = preloaded[String(sampleId)];
```

전통 문법 (실제 코드)	최신 문법
`pump` 함수가 끝날 때 다음 항목을 시작하는 재귀	`async` 작업자 세 개가 `while`로 대기열을 비움
`.then(성공, 실패)`	`try`와 `await`, 선택적 `catch` 바인딩
`ids || []`, `onStep || function () {}`	기본 매개변수

전통 문법 (실제 코드)	최신 문법
`new Promise`에 `setTimeout`으로 직접 시간 제한	`Promise.race`와 `AbortController`

7. 왜 이렇게

선택	이유
한 번에 3개씩 받습니다	브라우저가 같은 서버에 동시에 여는 연결 수가 제한되어 있습니다(보통 6개). 3개면 다른 요청(장치, 플러그인 목록)이 밀리지 않으면서 순차보다 빠릅니다.
로딩 화면은 `data`가 있을 때만 샘플을 받습니다	프로젝트를 읽지 못했으면 어떤 샘플이 필요한지 모릅니다.
`new` 모드에서는 샘플을 받지 않습니다	방금 만든 프로젝트라 샘플이 없습니다. 주소에 `&new=1`이 남은 채 새로고침하면 `new` 모드로 보이고 미리 받기를 건너뛰는데, 이 경우 샘플은 재생할 때 받으므로 기능에는 문제가 없고 로딩 문구만 "NEW PROJECT"로 보입니다.
미리 받은 것을 `loadSampleBytes`를 통해서만 꺼냅니다	재생(`PlaybackSession`)과 내보내기(`renderOffline`)가 이미 이 함수를 샘플 파일의 단일 출입구로 씁니다. 그 호출 쪽을 고치지 않고도 미리 받은 것이 쓰입니다.
오디오 컨텍스트는 여기서 만들지 않습니다	브라우저 정책상 사용자가 화면을 조작하기 전에는 `AudioContext`가 소리를 낼 수 없습니다. 목업의 결정("오디오는 시퀀서 안 첫 클릭이나 키 입력 때")대로, 컨텍스트는 재생을 누를 때 만들어집니다. 그래서 `AUDIO ENGINE` 단계는 실제 엔진을 켜는 일이 아니라 화면 준비를 기다리는 단계입니다(개요 90 §7).
Web Audio를 지원하지 않는 브라우저에서는 알림을 띄웁니다	열리기는 하지만 재생과 내보내기가 안 된다는 것을 미리 알려 줍니다.
로딩 화면이 있어도 `init`의 흐름은 그대로입니다	명령 레지스트리, 되돌리기, 저장, 단축키를 세우는 순서와 `loadProject`의 오류 처리를 바꾸지 않았습니다. 로딩 화면은 그 위에 얹은 층입니다.

8. 시험

`docs/handoff/s1-14_tests/ui_loader.mjs` (47건)가 실제 시퀀서를 헤드리스 Chrome으로 열고 60ms마다 로딩 화면의 상태를 기록합니다. 시험이 자기 앱 서버(8014, 서버용 Sentry는 끔)를 띄웁니다. 시험용 샘플 3개(`s1-14-a/b/c`)와 클립 3개(샘플 a를 둘이 씀)와 Sampler(샘플 c)를 만들고, 끝나면 지웁니다.

장면	확인한 것
A 기존 프로젝트 (샘플 받기를 각 0.7초 늦춤)	첫 표본부터 화면을 덮음, 종류 `OPENING PROJECT`, 정보 글(BPM, 박자, 트랙 수를 점으로 이은 한 줄), 퍼센트가 0에서 100까지 줄지 않고 감, 파형이 100에서 0으로 줄어듦, 샘플 개수가 `0/3`에서 `3/3`(중복 샘플은 한 번만 셈), 세 점이 차례로 켜짐, `SIGNAL IN`이 100%일 때 켜진 뒤에 열림, 끝나면 DOM에서 사라짐, 뒤 화면은 로딩 중 `inert`이고 끝나면 풀려 재생 버튼이 눌림, 샘플 파일을 각각 한 번만 받음
A 재생	재생을 누르면 미리 받아 둔 샘플을 넘겨받고(담아 둔 수 3에서 0), 서버에서 다시 받지 않음(받은 횟수 3 그대로)
B 샘플 하나 실패, 하나 2.5초 지연	열림(12초 안), 실패한 것도 끝난 것으로 세어 `3/3`, 느린 샘플을 기다림(열리기까지 2.5초 이상), `NO SIGNAL`에 머물지 않고 정상으로 `SIGNAL IN`, 받은 2개만 담아 둠
C 새 프로젝트 (`&new=1`)	`NEW PROJECT`, 단계 글, `... MASTER TRACK ONLY`, 샘플 개수 칸이 비고 샘플 파일을 받지 않음, 기존 프로젝트(A)보다 빨리 열림
D 예시 화면	`EXAMPLE SCREEN`, 제목 `Example Project`와 안내 `SAMPLE DATA, NOTHING IS SAVED`, 단계 글, 4초 안에 열림, 샘플 파일을 받지 않음

장면	확인한 것
E 움직임 줄이기	`is-reduced`가 붙고, 최소 표시 시간과 열기 대기가 없어 A보다 0.5초 넘게 빨리 끝남
F 프로젝트 읽기 실패 (`GET /full` 차단)	6초 안에 열림, `NO SIGNAL`을 그대로 두고(`is-failed`) 옆, 안내 알림이 뜸
G 안전장치	샘플을 60초 지연시켜 멈춘 상태에서, 안전장치 애니메이션이 30초 지연으로 걸려 있고, 30초가 지난 것으로 건너뛰면 숨김과 클릭 통과가 됨

- 스크린샷 7장은 `docs/screenshots/2026-10-21_s1-14_loader/`에 있습니다. 실제 화면이 목업과 같은 모양인 것을 확인했습니다.
- 시험 중 발견한 일은 두 가지입니다. 하나는 314에 적은 종류 문구가 비치는 문제입니다. 다른 하나는 시험의 정규식이었는데, 수정 도중 PowerShell이 역슬래시를 두 번 써서 정규식이 틀어졌고(`\\b`), 그 결과 "정하기 전 표본은 숨겨져 있다"는 확인이 엉뚱하게 실패했습니다. 코드의 오류가 아니라 시험의 오류였고 고쳤습니다.
- 기존 시험(S1-09부터 S1-13의 화면 시험)은 로딩 화면이 클릭을 막지 않도록 시험 도구가 기본으로 끕니다. 묶음 실행기로 전체를 다시 돌려 회귀가 없는 것을 확인합니다(개요 90 §6).